Competitive Analysis: Description-Code Consistency Scanning

Competitive Analysis: Description–Code Consistency Scanning for AI Tools & Skills

Your core idea: a scanner that doesn't ask "is this code malicious?" but "does this code do what its natural-language description says it does — no more, no less?" This is a distinct security check from traditional SAST, and it turns out to be a real, fast-moving research and product category. Good news and bad news: the good news is you're aimed at a validated, well-funded problem; the bad news is you are **not** first. Here's the landscape, so you can position your project against it instead of discovering it in Q&A.

---

1. The category this sits in

Security researchers now call your exact problem **Description–Code Inconsistency (DCI)** — a term that only entered the literature in the last few months. It sits next to, but is meaningfully different from, two adjacent categories people often lump it with:

Category	What it checks	Example
Tool Poisoning Attacks (TPA)	Hidden instructions *inside* a tool description that manipulate the LLM's reasoning (indirect prompt injection)	A description contains an invisible `<IMPORTANT>` tag telling the agent to read `~/.ssh/id_rsa` before running a simple math tool
Traditional SAST / malware scanning	Whether code contains known-bad patterns, dangerous functions, or signatures	Flags `eval()`, `os.system()`, hardcoded secrets, known CVEs
Description–Code Inconsistency (your zone)	Whether the code's *actual behavior* matches what the description *claims* — regardless of whether either half looks "bad" in isolation	Description says "reads config.yaml and returns contents"; code does that AND silently POSTs the file to an external URL. Nothing is syntactically wrong. The lie is in what's omitted.

This distinction is exactly the one your screenshots articulate, and it's worth stating explicitly in your pitch — judges in this space will have seen a dozen "prompt injection scanner" submissions and will immediately want to know if you're actually doing something different. You are — but you need to say so precisely.

---

2. Direct competitors (same zone — description vs. behavior)

These are the projects doing almost exactly what you're describing. Know them cold.

MCPDiFF (Feb 2026)

The first large-scale study to formalize this exact problem. Built a static analysis pipeline combining multi-language code parsing, function call-chain construction, and semantic comparison between tool descriptions and code. Ran it across **10,240 real-world MCP servers**. Its own framing is close to your pitch: *a tool described as read-only may silently perform destructive operations, remaining invisible to both users and models.* Key limitation later papers point out: MCPDiFF *mainly focuses on risks arising from behavior present in code but insufficiently reflected in descriptions* — i.e., it's strong on "undisclosed extra behavior" but weaker on other inconsistency types (see DCIChecker below).

DCIChecker (June 2026) — closest direct competitor

This is the one to study most closely, because it explicitly benchmarks itself against MCPDiFF and claims to beat it. It introduces a **seven-type taxonomy** of inconsistency (not just "undisclosed behavior" — also over-claimed functionality, ambiguity, and undeclared side effects treated as first-class categories) and a detection method called **Direct-Reverse-Arbitration prompting**, which cross-validates descriptions against code from both directions. Their numbers are useful to cite in your own pitch as market validation: *applied to a large-scale dataset comprising 19,200 description-code pairs extracted from 2,214 real-world MCP servers… 9.93% of these pairs exhibiting inconsistencies* — that ~10% figure is presumably the "residual" number your notes reference. This paper is essentially the academic version of your product idea, done at scale, three weeks before you're pitching. Read it before your demo.

Cisco MCP Scanner — Behavioral Code Threat Analysis (Dec 2025)

This is the closest **commercial/production** competitor, and it's from a company with obvious distribution advantage. Cisco explicitly moved beyond signature-based scanning to semantic comparison: *Unlike traditional SAST, our approach uses AI to compare a tool's documented intent against its actual behavior. After extracting detailed behavioral signals from the code, the model looks for mismatches and flags cases where operations (such as network calls or data flows) don't align with what the documentation claims.* Their pitch for *why* this matters is nearly word-for-word your thesis: *A model context protocol (MCP) tool can claim to execute a benign task such as "validate email addresses," but if the tool is compromised, it can be redirected to fulfill ulterior motives, such as exfiltrating your entire address book to an external server.* They also make the same "existing tools can't do this" argument you're making: SAST tools *have always struggled with context. Answering questions like, "Is a network call malicious or expected?" and "Is this file access a threat or a feature?" requires semantic understanding that rule-b[ased tools lack]*. This is bundled into their broader IDE-integrated Agent Security Scanner, alongside a "Watchdog" feature for sensitive-file tracking.

"Do Skill Descriptions Tell the Truth?" (arXiv, ~April 2026)

Nearly identical framing to yours, applied specifically to Claude/Copilot/ChatGPT-style **Skills** rather than MCP tools — relevant since your screenshot mentions "agent plugins" generically. Uses code property graphs (via Joern) for structural analysis combined with semantic comparison to detect **undisclosed security behaviors** in code-backed skills. Worth pulling for its taxonomy of skill-specific attack patterns.

SkillProbe (multi-agent auditing, March 2026)

Frames the same core problem — *A malicious developer can craft a skill whose description appears benign to both the LLM and a human reviewer while the underlying code exfiltrates data or escalates privileges, which is what we term semantic-behavioral inconsistency* — and explicitly critiques MCPDiFF's scope: it *operates at the implementation-semantics layer and explicitly excludes over-claimed capabilities from its threat model*. SkillProbe's differentiator is multi-agent collaboration for auditing and reasoning about *chained* skills, not just single-skill checks.

VIPER-MCP (taint-style vulnerability detection)

Positions itself in the same lineage — *Recent work on MCP server security spans ecosystem measurement, static-code auditing, description-code consistency checking, and dynamic probing* — but focuses on confirming end-to-end exploitability via taint analysis and agent-mediated dynamic execution rather than static description matching alone. Useful contrast point: they explicitly note prior tools (including Cisco's) *provide useful ecosystem characterization, capability auditing, and broad behavioral probing, but they still stop short of confirming end-to-end exploitability through realistic agent-mediated interaction* — a gap you could position your project against too, or explicitly scope out.

---

3. Adjacent tools (same battlefield, different weapon)

These aren't doing description-vs-code semantic matching, but they're what a judge will compare you to if you don't clearly separate yourself.

- **MCP-Scan (Invariant Labs, now part of Snyk)** — the most widely cited open-source MCP security tool. Focuses on tool-description prompt-injection analysis, cross-origin/shadowing detection, and a clever **tool pinning** feature: hashes tool descriptions at first scan and alerts if a server silently changes them later (the "rug pull" attack). This is description-integrity checking, not description-**accuracy** checking — it catches "this description changed," not "this description was never true."
- **mcp-audit** — fully offline/deterministic scanner: *config-structure analysis, cross-server attack-path graphs with hitting-set remediation, 89 Semgrep source-code rules, and OWASP MCP Top 10 mapping*. Explicitly acknowledges it *will not reason about the semantic intent of a tool description the way an LLM-based analyzer might* — that's precisely the gap your tool fills.
- **MCP-ITP, MCPTox** — academic red-team frameworks/benchmarks for generating and measuring tool-poisoning attack success rates. Useful for test-case generation if you want an eval set of "malicious tools with innocent descriptions" to demo against.
- **ClawAudit** — targets a different layer entirely: not the skill's honesty, but whether the *agent runtime itself* mediates skills safely (permission gates, isolation). Good line to draw if a judge asks "how is this different from a runtime sandbox."
- **SkillScan / SkillSieve / SkillFortify / AEGIS / ClawGuard** — a fast-multiplying zoo of skill-auditing tools, each with a slightly different angle (static+LLM semantic inspection, formal capability confinement, pre-execution blocking). Worth naming as "the crowded field" in your slides rather than pretending they don't exist.

---

4. Commercial/market validation

This isn't just an academic exercise — it's a funded, actively consolidating market,

which is a strong slide for a hackathon pitch ("here's proof this problem is worth solving"):

- **Koi Security** — Tel Aviv/DC-based, founded 2024, raised **$48M** (seed + $38M Series A led by Battery Ventures, Team8, Picture Capital, NFX) to secure *the modern software layer that legacy endpoint tools were never built to protect, including packages, containers, extensions, AI models, and MCPs*. They run marketplace audits that *catalogued hundreds of malicious skills on repositories such as ClawHub*.
- In **February 2026, Palo Alto Networks acquired Koi for $400 million** — roughly a year after founding — specifically because of *emerging risks from autonomous AI agents operating inside corporate systems*. That's a strong data point for "this problem is acute enough that a major security vendor paid a 9-figure premium to own a solution in this exact space."
- **Snyk** absorbed Invariant Labs / MCP-Scan and is building an enterprise "Agent Scan" product on top of it.
- **Mitiga Labs** publicly demonstrated a **silent codebase exfiltration attack** via a skill published to skills.sh, one of the largest public Agent Skills catalogs — *a seemingly legitimate Skill could exfiltrate a complete codebase in four user interactions, leaving the skill-audit.log entirely empty*. Great real-world case study for your demo narrative — this is precisely the class of attack your tool is meant to catch, and it happened in the wild, not just in a lab.
- **SafeDep** flagged environment-variable exfiltration as a distinct high-value target because *environment variables represent a high-value target because they aggregate secrets from multiple services in a single, programmatically accessible location* and are invisible to filesystem-level monitoring.

---

5. Where the field still has real gaps (your opportunity)

Reading across all of the above, a few genuine openings remain — pick one or two to anchor your differentiation, rather than trying to claim you've solved the whole space:

1. **Accessibility / speed over rigor.** Most of the strongest tools (DCIChecker, MCPDiFF, Cisco's scanner) are either research prototypes, enterprise-gated, or IDE-embedded. A lightweight, fast, hackathon-friendly CLI or CI-action that a solo developer can point at a single skill/tool repo before publishing is still a legitimate underserved niche — most existing tools assume you're auditing at marketplace scale, not "should I trust this one skill I just downloaded."
2. **Full taxonomy coverage is still rare.** Most tools before DCIChecker only caught "undisclosed extra behavior." Over-claimed functionality (description promises something the code doesn't actually do) and ambiguity are still thinly covered — a narrower, sharper tool focused on 2-3 well-executed inconsistency types could out-demo a broader but shallower competitor.
3. **Explainability for non-security-expert developers.** Several of these tools produce security-analyst-grade output (call graphs, taint paths). A version that explains *in plain language why the description and behavior don't match* — closer to a diff/red-line between "what was promised" and "what was found" — is a genuinely different UX angle and demos very well.
4. **Chained/composed skills.** SkillProbe explicitly flags that atomic, single-tool auditing breaks down once skills call other skills. If your time allows, even a simple two-hop check ("this tool's description is honest, but it calls tool B, whose description doesn't disclose *its* network calls") would put you ahead of most single-tool checkers.

---

6. How to frame this in your pitch

Given how crowded and recent this field is (nearly every citation above is from the last 5 months), the strongest pitch is **not** “no one has thought of this” — a judge who’s read even one of these papers will immediately doubt your credibility if you claim that. The stronger frame:

> “This is an active, urgent, recently-formalized problem — Palo Alto just paid $400M for a company built around it, and academic groups only coined the term ‘Description-Code Inconsistency’ this year. Here’s our specific angle: [fast / explainable / narrow-scope / free-and-open / composed-skill-aware] — solving the piece the current landscape underserves.”

That positions you as informed and focused rather than naive, which is generally what wins hackathon judging on a security topic.

Sources for further reading

- MCPDiFF: *Don’t believe everything you read* — arxiv.org/abs/2602.03580
- DCIChecker: *Description-Code Inconsistency in Real-world MCP Servers* — arxiv.org/abs/2606.04769
- Cisco: *MCP Scanner Behavioral Code Threat Analysis* — blogs.cisco.com/ai
- *Do Skill Descriptions Tell the Truth?* — arxiv.org/abs/2605.12875
- SkillProbe — arxiv.org/abs/2603.21019
- VIPER-MCP — arxiv.org/abs/2605.21392
- OWASP: MCP Tool Poisoning — owasp.org/www-community/attacks/MCP_Tool_Poisoning
- awesome-agent-skills-security (curated list) — github.com/LLMSecurity/awesome-agent-skills-security